US 20250284596A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0284596 A1**

Thatikonda et al. (43) **Pub. Date: Sep. 11, 2025**

(54) **OPTIMIZING LARGE DATABASE BACKUP TRANSACTIONS ACROSS MULTI SUBSTATE CLOUD ENVIRONMENTS FOR IMPROVED DATABASE SYSTEMS AVAILABILITY**

(71) Applicant: **Salesforce, Inc.**, San Francisco, CA (US)

(72) Inventors: **Kalyan Chakravarthy Thatikonda**, San Francisco, CA (US); **Arul Tamilmani**, San Francisco, CA (US)

(73) Assignee: **Salesforce, Inc.**, San Francisco, CA (US)

(21) Appl. No.: **18/596,450**

(22) Filed: **Mar. 5, 2024**

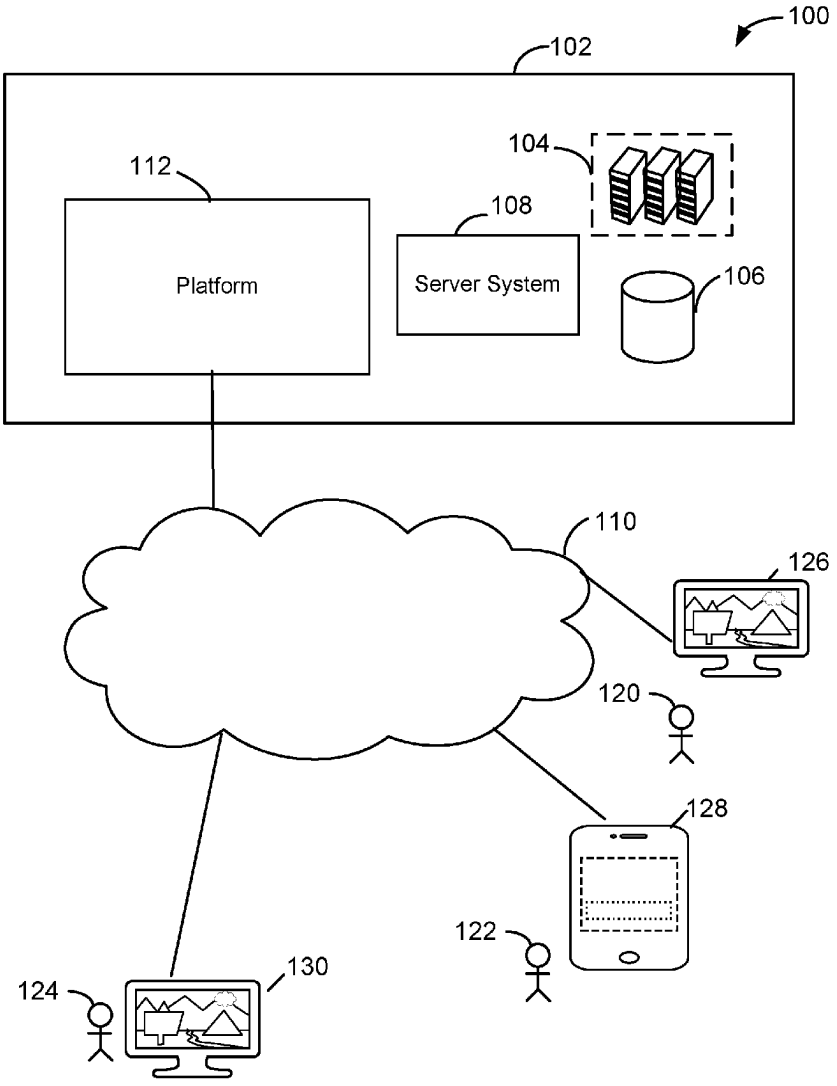
Publication Classification

(51) **Int. Cl.**
G06F 11/14 (2006.01)

(52) **U.S. Cl.**
CPC *G06F 11/1458* (2013.01); *G06F 2201/80* (2013.01)

(57) **ABSTRACT**

Disclosed are some implementations of systems, apparatus, methods and computer program products for optimizing database backup transactions. A backup poller in a private subnet tracks a database backup operation and detects a failure of the database backup operation. More particularly, the backup poller detects that a failure occurred at a specific point in the database backup operation. The backup poller instructs a backup decentralizer in a public subnet to continue the database backup operation from the specific point. The backup decentralizer monitors a health of a plurality of service providers and selects a service provider of the plurality of service providers based on the health of the service provider. The backup decentralizer continues the backup operation from the specific point using the selected service provider.



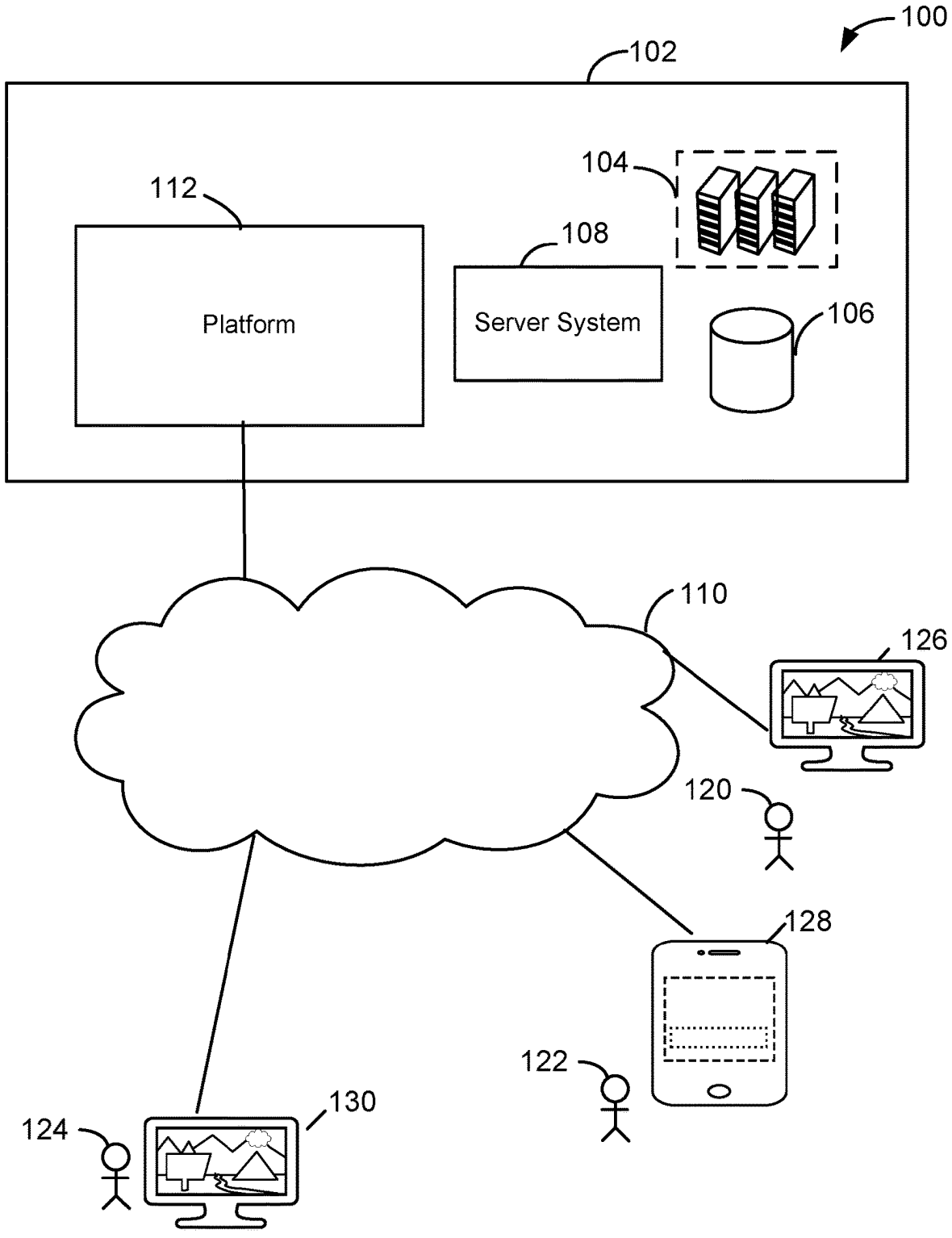

100
102
104
112
108
Platform
Server System
106
110
126
120
128
122
124
130

Figure 1

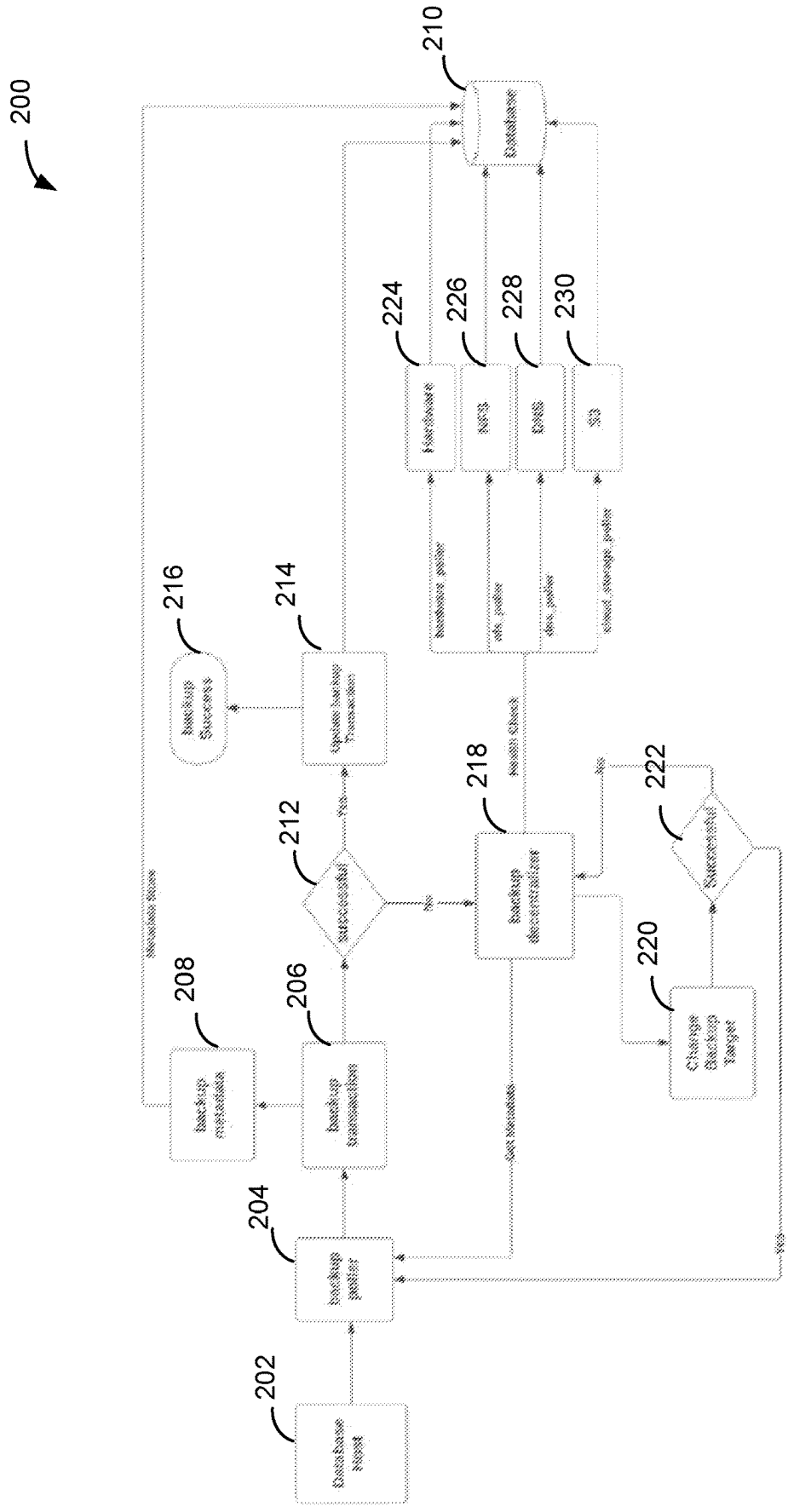


Figure 2

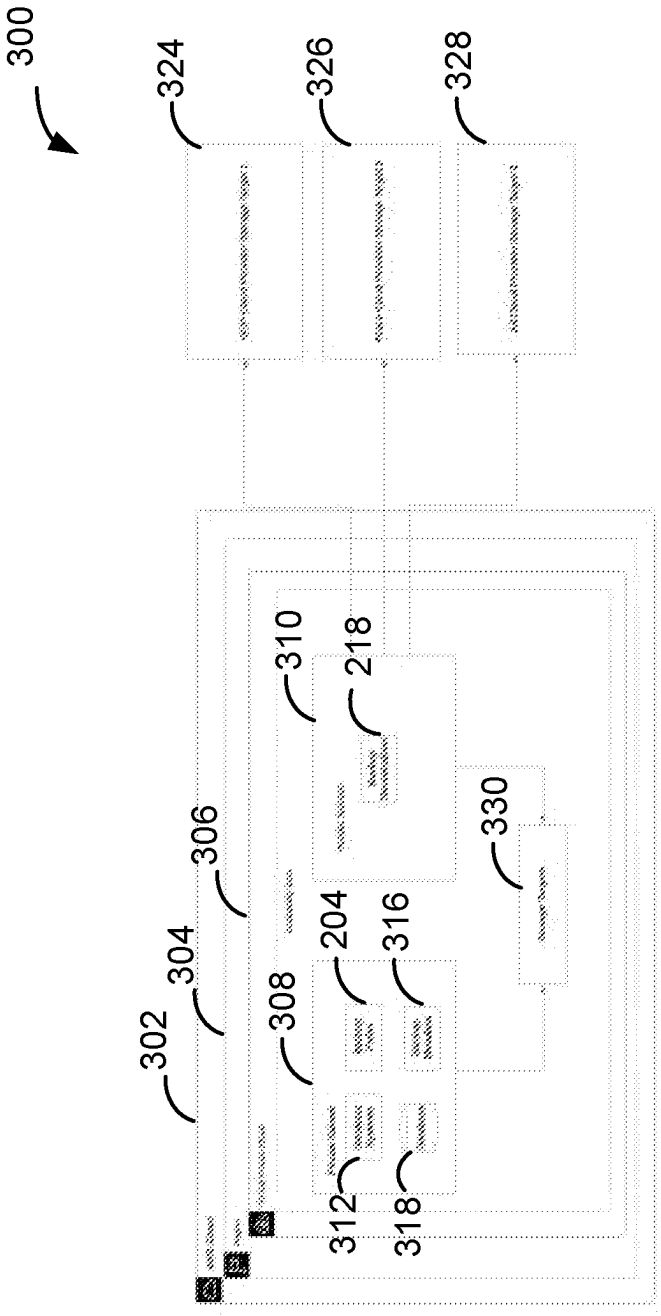


Figure 3

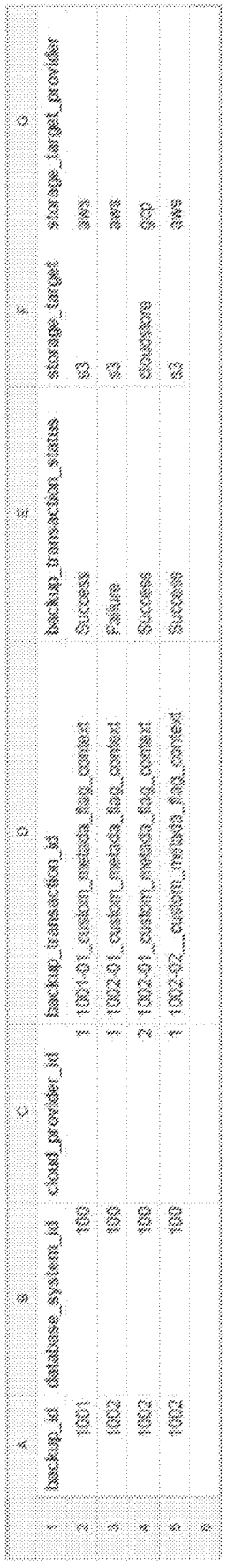

400
A
B
C
D
E
F
G
backup_id
database_system_id
cloud_provider_id
backup_transaction_id
backup_transaction_status
storage_target
storage_target_provider
1001
100
1
1001-01_custom_metada_tag_content
Success
s3
aws
1002
100
1
1002-01_custom_metada_tag_content
Failure
s3
aws
1002
100
2
1002-01_custom_metada_tag_content
Success
cloudstore
gcp
1002
100
1
1002-02_custom_metada_tag_content
Success
s3
aws

Figure 4A


450
A
B
C
D
1
storage_target_id
storage_provider
storage_target_name
storage_health_target
2
S001
aws
s3 cloud storage
healthy
3
S002
gcp
google cloud storage
healthy
4
S003
azure
azure cloud storage
healthy
5
S004
ali
ali cloud storage
healthy
6

Figure 4B

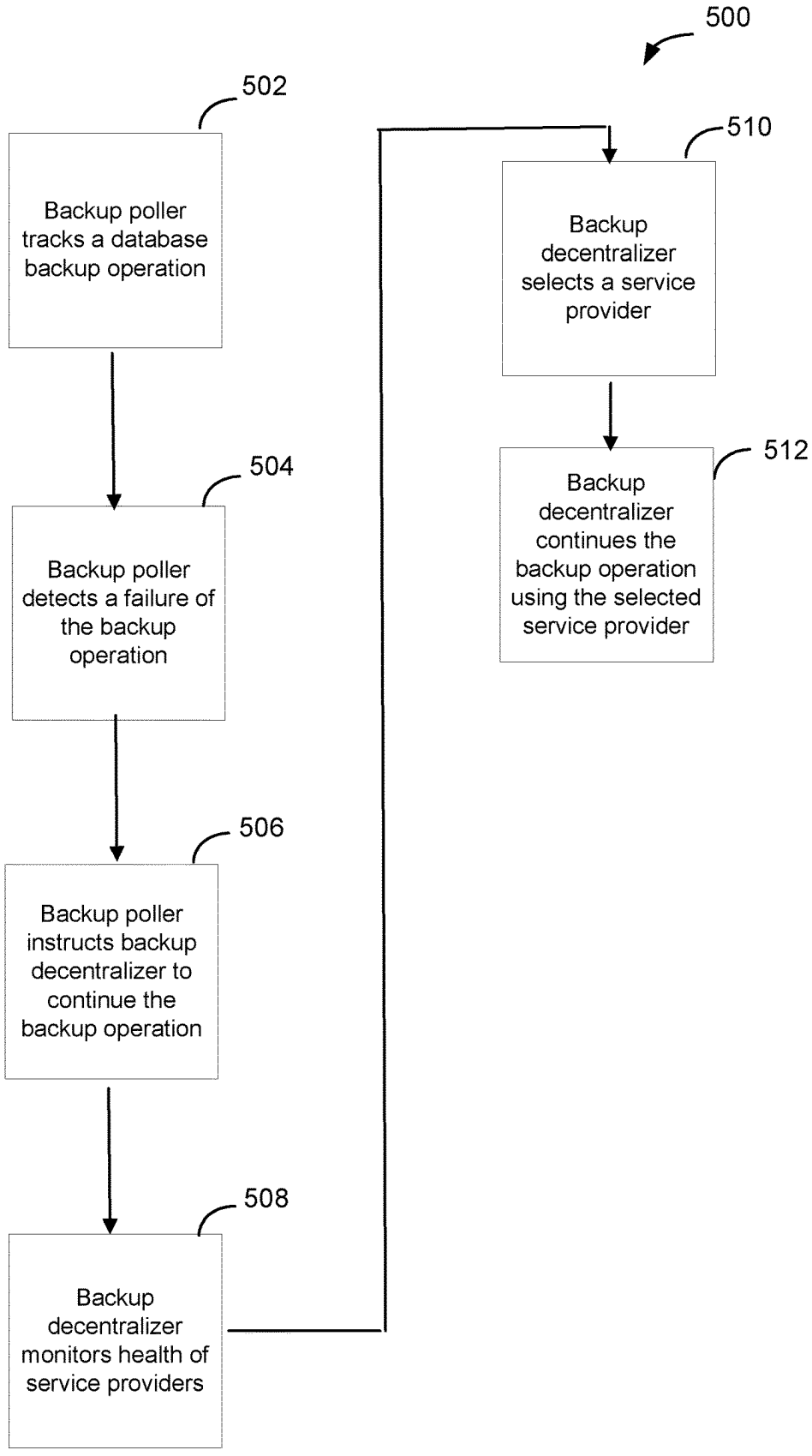

500
502
Backup poller tracks a database backup operation
504
Backup poller detects a failure of the backup operation
506
Backup poller instructs backup decentralizer to continue the backup operation
508
Backup decentralizer monitors health of service providers
510
Backup decentralizer selects a service provider
512
Backup decentralizer continues the backup operation using the selected service provider

Figure 5

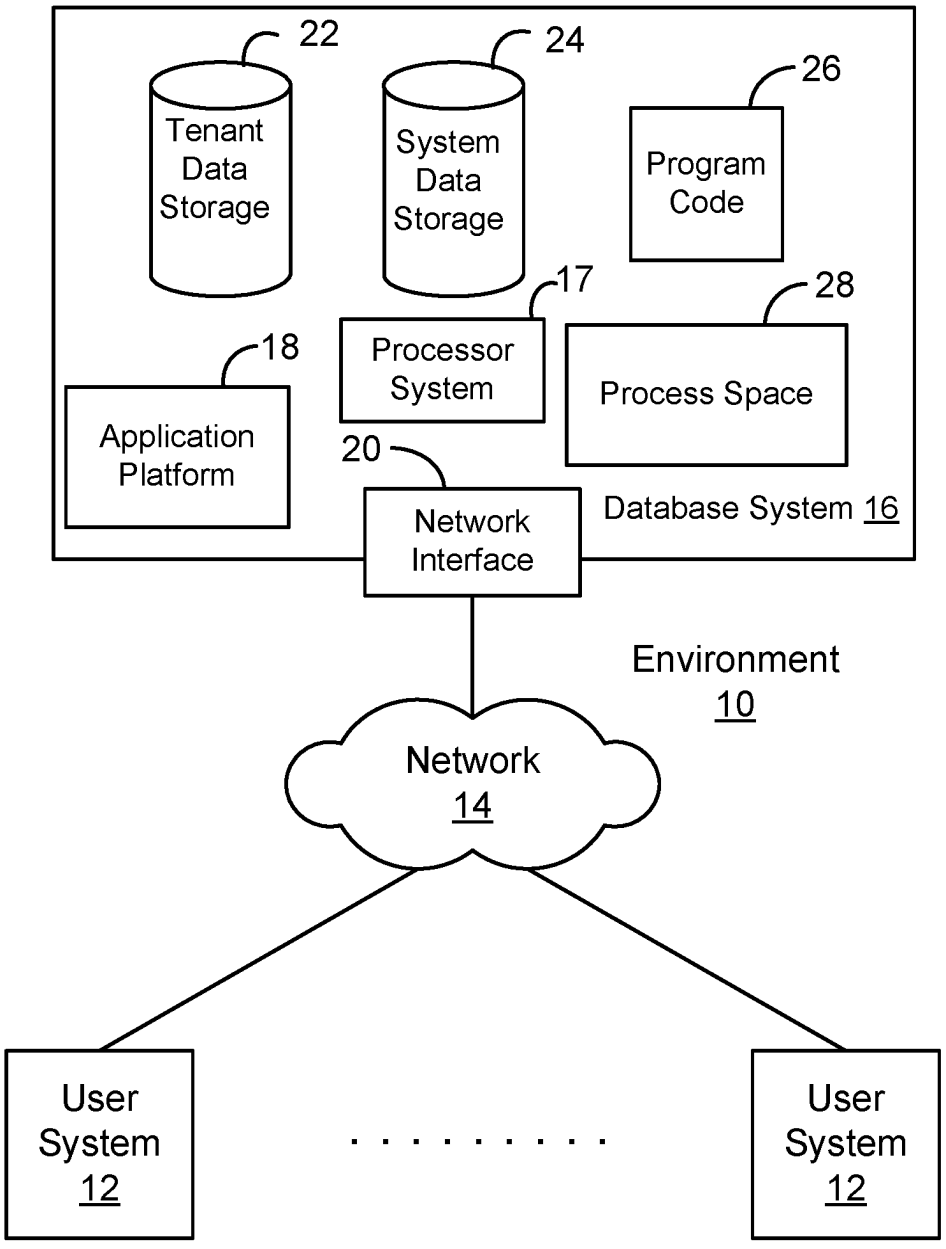

22
24
26
Tenant Data Storage
System Data Storage
Program Code
17
28
18
Processor System
Process Space
Application Platform
20
Network Interface
Database System 16
Environment 10
Network 14
User System 12
User System 12

Figure 6A

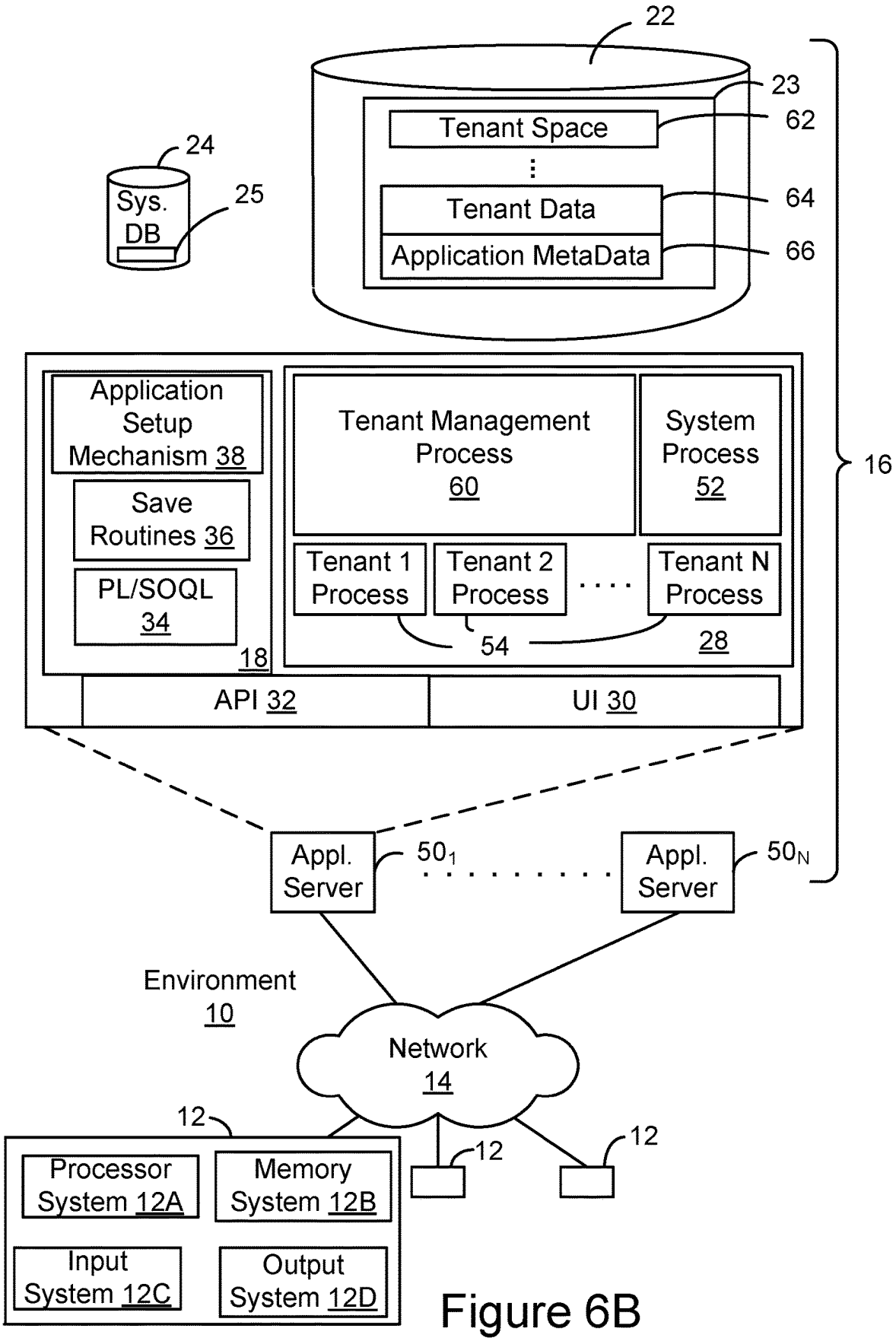

22
23
Tenant Space
62
24
Sys. DB
25
Tenant Data
64
Application MetaData
66
Application Setup Mechanism 38
Save Routines 36
PL/SOQL 34
18
Tenant Management Process 60
System Process 52
Tenant 1 Process
Tenant 2 Process
Tenant N Process
54
28
16
API 32
UI 30
Appl. Server
50₁
Appl. Server
50N
Environment 10
Network 14
12
12
12
Processor System 12A
Memory System 12B
Input System 12C
Output System 12D

Figure 6B

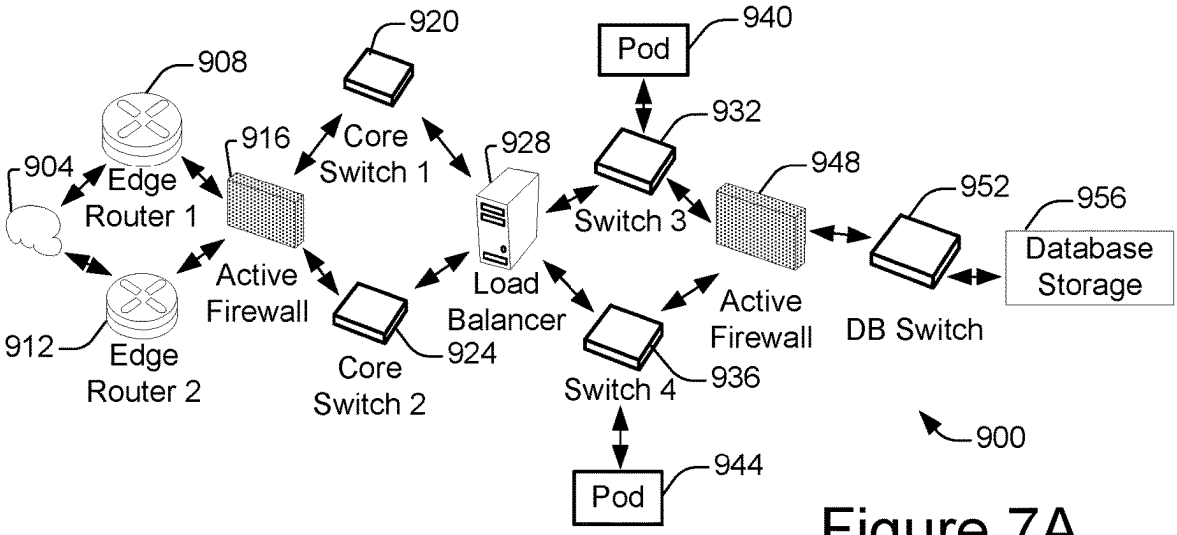


Figure 7A

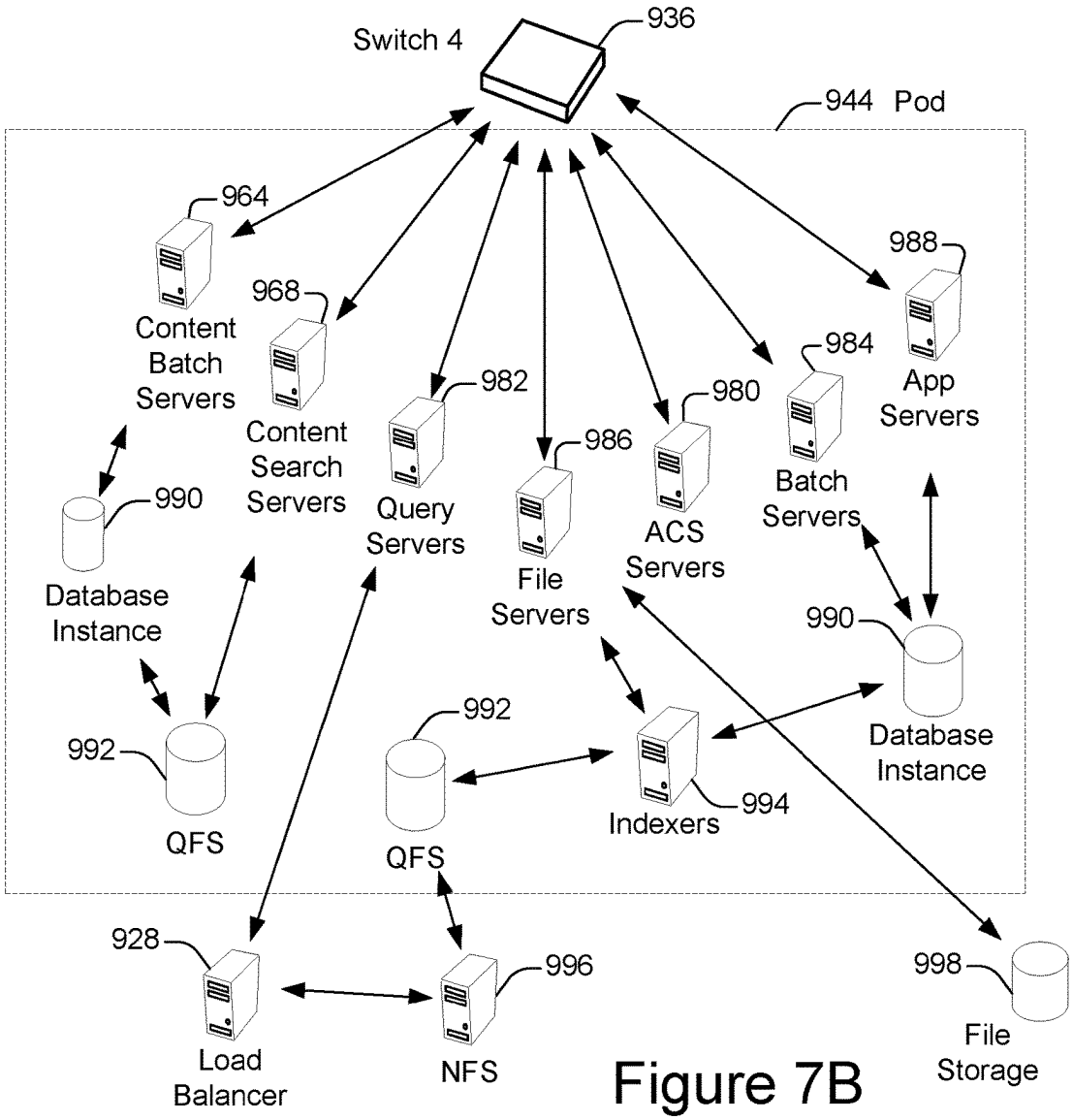


Figure 7B

OPTIMIZING LARGE DATABASE BACKUP TRANSACTIONS ACROSS MULTI SUBSTATE CLOUD ENVIRONMENTS FOR IMPROVED DATABASE SYSTEMS AVAILABILITY

COPYRIGHT NOTICE

FIELD OF TECHNOLOGY

[0002] This patent document relates generally to systems and techniques associated with database backup operations, and more specifically to performing database backup operations across multiple environments.

BACKGROUND

[0003] "Cloud computing" services provide shared resources, applications, and information to computers and other devices upon request. In cloud computing environments, services can be provided by one or more servers accessible over the Internet rather than installing software locally on in-house computer systems. Users can interact with cloud computing services to undertake a wide range of tasks.

[0004] Database backup solutions help businesses protect their data in the event of corruption, user error, or hardware failure. With database backup software, it is possible to automate the backup of data.

BRIEF DESCRIPTION OF THE DRAWINGS

[0005] The included drawings are for illustrative purposes and serve only to provide examples of possible structures and operations for the disclosed inventive systems, apparatus, methods and computer program products for facilitating the backup of data. These drawings in no way limit any changes in form and detail that may be made by one skilled in the art without departing from the spirit and scope of the disclosed implementations.

[0006] FIG. **1** shows a diagram of an example of a system **100** in which database backup operations are implemented, in accordance with some implementations.

[0007] FIG. **2** shows an example of the operation of a system including a backup poller and backup decentralizer, in accordance with some implementations.

[0008] FIG. **3** shows a diagram of an example of a system including a backup poller and backup decentralizer, in accordance with some implementations.

[0009] FIG. **4**A shows an example of information maintained by the backup poller, in accordance with some implementations.

[0010] FIG. **4**B shows an example of information maintained by the backup decentralizer, in accordance with some implementations.

[0011] FIG. **5** shows a process flow diagram illustrating an example of a process for implementing database backup operations, in accordance with some implementations.

[0012] FIG. **6**A shows a block diagram of an example of an environment **10** in which an on-demand database service can be used in accordance with some implementations.

[0013] FIG. **6**B shows a block diagram of an example of some implementations of elements of FIG. **6**A and various possible interconnections between these elements.

[0014] FIG. **7**A shows a system diagram of an example of architectural components of an on-demand database service environment **900**, in accordance with some implementations.

[0015] FIG. **7**B shows a system diagram further illustrating an example of architectural components of an on-demand database service environment, in accordance with some implementations.

DETAILED DESCRIPTION

[0016] Examples of systems, apparatus, methods and computer program products according to the disclosed implementations are described in this section. These examples are being provided solely to add context and aid in the understanding of the disclosed implementations. It will thus be apparent to one skilled in the art that implementations may be practiced without some or all of these specific details. In other instances, certain operations have not been described in detail to avoid unnecessarily obscuring implementations. Other applications are possible, such that the following examples should not be taken as definitive or limiting either in scope or setting.

[0017] In the following detailed description, references are made to the accompanying drawings, which form a part of the description and in which are shown, by way of illustration, specific implementations. Although these implementations are described in sufficient detail to enable one skilled in the art to practice the disclosed implementations, it is understood that these examples are not limiting, such that other implementations may be used and changes may be made without departing from the spirit and scope of the disclosure.

[0018] The described subject matter may be implemented in the context of any computer-implemented system, such as a software-based system, a database system, a multi-tenant environment, or the like. Moreover, the described subject matter may be implemented in connection with two or more separate and distinct computer-implemented systems that cooperate and communicate with one another. One or more examples may be implemented in numerous ways, including as a process, an apparatus, a system, a device, a method, a computer readable medium such as a computer readable storage medium containing computer readable instructions or computer program code, or as a computer program product comprising a computer usable medium having a computer readable program code embodied therein.

[0019] Any of the disclosed implementations may be used alone or together with one another in any combination. The one or more implementations encompassed within this specification may also include examples that are only partially mentioned or alluded to or are not mentioned or alluded to at all in this brief summary or in the abstract. Although various implementations may have been motivated by various deficiencies with the prior art, which may be discussed or alluded to in one or more places in the specification, the implementations do not necessarily address any of these deficiencies. In other words, different implementations may address different deficiencies that may

be discussed in the specification. Some implementations may only partially address some deficiencies or just one deficiency that may be discussed in the specification, and some implementations may not address any of these deficiencies.

[0020] Some implementations of the disclosed systems, apparatus, methods and computer program products are configured to facilitate the process of performing database backups In some implementations, a database system is implemented using a backup poller configured to monitor database backup operations and a backup decentralizer configured to monitor the health of service providers that can perform database backup operations, enabling database backup operations to be processed on different targets across different cloud environments.

[0021] Interruptions on critical database backup transactions can cause database nodes to be in a hung state, resulting in the database cluster to be down even though the underlying database system components are healthy. This can be a serious problem if it happens at scale across multiple clouds on different database clusters, which could result in performance degradation or outage.

[0022] Unfortunately, it is impossible to resume the state of a backup transaction on a different storage target due to platform dependencies. Furthermore, it is impossible to decentralize backup transactions in real-time due to the dependencies on the health of the storage targets, as well as the hardware. Further problems result from network issues on the database hosts hosted in a public cloud during the database backup transactions. Furthermore, there is a lack of transparency of the database backup transactions to determine corrective actions.

[0023] FIG. **1** shows a diagram of an example of a system **100** in which database backup operations are implemented, in accordance with some implementations. Database system **102** includes a variety of different hardware and/or software components that are in communication with each other. In the non-limiting example of FIG. **1**, system **102** includes any number of computing devices such as servers **104**. Servers **104** are in communication with one or more storage mediums **106** configured to store and maintain relevant data and/or metadata used to perform some of the techniques disclosed herein, as well as to store and maintain relevant data and/or metadata generated by the techniques disclosed herein. Storage mediums **106** may further store computer-readable instructions configured to perform some of the techniques described herein. Storage mediums **106** can also store user profiles, layouts, and/or database records such as customer relationship management (CRM) records.

[0024] System **102** includes server system **108** that facilitates the database backup process, as described herein. As will be described in further detail below, server system **108** can include a backup poller and backup decentralizer, as described herein.

[0025] In some implementations, system **102** is configured to store user profiles/user accounts associated with users of system **102**. Information maintained in a user profile of a user can include a client identifier such an Internet Protocol (IP) address or Media Access Control (MAC) address. In addition, the information can include a unique user identifier such as an alpha-numerical identifier, the user's name, a user email address, and credentials of the user. Credentials of the user can include a username and password. The information can further include job related information such as a job title, role, group, department, organization, and/or experience level, as well as any associated permissions. Job related information and any associated permissions can be applied by server system **108**.

[0026] Client devices **126**, **128**, **130** may be in communication with system **102** via network **110**. More particularly, client devices **126**, **128**, **130** may communicate with servers **104** via network **110**. For example, network **110** can be the Internet. In another example, network **110** comprises one or more local area networks (LAN) in communication with one or more wide area networks (WAN) such as the Internet.

[0027] Embodiments described herein are often implemented in a cloud computing environment, in which network **110**, servers **104**, and possible additional apparatus and systems such as multi-tenant databases may all be considered part of the "cloud." Servers **104** may be associated with a network domain, such as www.salesforce.com and may be controlled by a data provider associated with the network domain. In this example, employee users **120**, **122**, **124** of client computing devices **126**, **128**, **130** have accounts at salesforce.com®. By logging into their accounts, users **126**, **128**, **130** can access the various services and data provided by system **102** to employees. In other implementations, users **120**, **122**, **124** need not be employees of salesforce.com® or log into accounts to access services and data provided by system **102**. Examples of devices used by users include, but are not limited to a desktop computer or portable electronic device such as a smartphone, a tablet, a laptop, a wearable device such as Google Glass®, another optical head-mounted display (OHMD) device, a smart watch, etc.

[0028] In some implementations, users **120**, **122**, **124** of client devices **126**, **128**, **130** can access services provided by system **102** via platform **112** or an application installed on client devices **126**, **128**, **130**. More particularly, client devices **126**, **128**, **130** can log into system **102** via an application programming interface (API) or via a graphical user interface (GUI) using credentials of corresponding users **120**, **122**, **124** respectively.

[0029] Some implementations may be described in the general context of computing system executable instructions, such as program modules, being executed by a computer. Generally, program modules include routines, programs, objects, components, data structures, etc. that performs particular tasks or implement particular abstract data types. Those skilled in the art can implement the description and/or figures herein as computer-executable instructions, which can be embodied on any form of computing machine program product discussed below.

[0030] Some implementations may also be practiced in distributed computing environments where tasks are performed by remote processing devices that are linked through a communications network. In a distributed computing environment, program modules may be located in both local and remote computer storage media including memory storage devices.

[0031] FIG. **2** shows an example of the operation of a system **200** including a backup poller and backup decentralizer, in accordance with some implementations. In some implementations, a database host **202** communicates with a backup poller **204**. More particularly, database host **202** may initiate a backup of a database system by communicating with backup poller **204**. Backup poller **204** may be configured to initiate and/or monitor database backup transaction

(s) **206** operating on a backup storage target. Specifically, backup poller **204** may access backup metadata **208** and backup metadata **208** associated with backup transaction **206** may be updated in database **210**. Backup poller **204** may determine whether backup transaction **206** is successful at **212**. If backup transaction **206** is successful, this status may be updated in the backup metadata **208** in database **210** and the process ends with a successful backup **216**.

[0032] The backup metadata **208** can include information including, but not limited to, a backup identifier (ID), a backup file name, a backup file ID, backup file type, backup file size, backup assigned initial storage provider ID, backup cloud provider, backup cloud provider ID, backup cloud provider storage class, backup cloud provider storage class ID, backup file ID transaction status, an indicator of backup file size, backup file ID split file ID, backup file ID split file ID transaction status, backup retry limit per cloud provider, and/or a backup retry timeout.

[0033] If backup transaction **206** is not successful at **212**, backup poller **204** may communicate with backup decentralizer **218**, which may operate to change the backup target at **220**. More particularly, backup decentralizer **218** may perform health checks to identify and select a healthy target (e.g., service provider). As shown in this example, backup decentralizer may perform a health check by polling hardware **224**, network file system (NFS) **226**, domain name system (DNS) **228**, and/or storage service S3 **230**. Once successfully changed at **222**, backup poller **204** continues to operate for further backup transactions.

[0034] FIG. **3** shows a diagram of an example of a system **300** including a backup poller and backup decentralizer, in accordance with some implementations. Within cloud **302**, a specific region **304** may include a virtual private cloud **306**. Within virtual private cloud **306** is a private subnet **308** and public subnet **310**. Within private subnet **308** is backup poller **204** and related systems such as database systems **312**, backup metadata **316** and database **318** storing backup metadata **316**. Public subnet **310** includes backup decentralizer **218**. By implementing backup decentralizer **218** in a public cloud, this enables backup decentralizer **218** to communicate with various service providers associated with various storage targets **324**, **326**, **328**. Backup poller **204** and/or backup decentralizer **218** may communicate with one or more storage targets **330**, which can include storage targets **324**, **326**, **328**. More particularly, backup decentralizer **218** can include a public application programming interface (API) that performs a health check on system services that are or will be engaged in a database backup transaction. When invoked, the backup decentralizer **218** retrieves the metadata from backup poller **204** and resumes the backup after performing health checks to change the backup target. This allows the backup to be resumed on a different target.

[0035] FIG. **4**A shows an example of information **400** maintained by the backup poller, in accordance with some implementations. Information **400** can include database table(s) that store information backup metadata such as the status of database backup transactions. In this example, a database table stores, for a given entry, a backup identifier identifying a particular backup process, database system identifier identifying a database system, cloud provider identifier identifying a cloud, backup transaction identifier identifying a specific backup transaction, backup transaction status (e.g., success or failure), storage target in which data is being stored, and storage target provider identifying a service provider.

[0036] FIG. **4**B shows an example of information **450** maintained by the backup decentralizer, in accordance with some implementations. More particularly, information **450** can include the health of cloud storage service providers. In this example, a storage health table includes a storage target identifier, a storage provider, a storage target name, and storage health indicator (e.g., healthy or unhealthy).

[0037] FIG. **5** shows a process flow diagram illustrating an example of a process **500** for implementing database backup operations, in accordance with some implementations. A backup poller in a private subnet tracks a database backup operation at **502**. More particularly, the backup poller monitors a backup transaction from start to end and alerts the backup decentralizer in the event of a backup failure for a particular backup transaction.

[0038] The backup poller maintains an audit of every backup transaction and extracts the metadata from each backup transaction, as detailed above. The backup poller may determine metadata associated with the database backup operation and store the metadata. The metadata may be persisted in a centralized database.

[0039] The backup poller determines whether the database backup operation was successful. The backup poller detects a failure of the database backup operation at a specific point in the database backup operation at **504**. The backup poller then instructs a backup decentralizer in a public subnet to continue the database backup operation from the specific point at **506**. More particularly, the backup poller may provide the metadata to the backup decentralizer. For example, the backup poller may provide at least a portion of the metadata to the backup decentralizer via an API associated with the backup decentralizer.

[0040] The backup decentralizer monitors a health of a plurality of service providers at **508**. The backup decentralizer selects a service provider of the plurality of service providers based on the health of the service provider at **510**. The backup decentralizer inherits the metadata associated with the backup transaction from the backup poller and safely routes the backup transaction to a different target. The backup decentralizer continues the backup operation from the specific point using the selected service provider at **512**. In this manner, the backup decentralizer ensures the continuity of the backup transaction across hybrid and public cloud environments.

[0041] The database backup architecture optimizes backup transactions and improves database availability on storage targets across multiple backup environments (e.g., public cloud, private cloud, hybrid) using a metadata driven approach and helps to decentralize storage targets for backup transactions across different cloud environments. Advantageously, the database backup architecture eliminates the storage platform dependencies.

[0042] In some implementations, a large backup transaction file may be split across multiple environments (e.g., public cloud, hybrid cloud, physical data center. Advantageously, dependencies on third party tools may be eliminated.

[0043] Some but not all of the techniques described or referenced herein are implemented using or in conjunction with a database system. Salesforce.com, inc. is a provider of customer relationship management (CRM) services and

other database management services, which can be accessed and used in conjunction with the techniques disclosed herein in some implementations. In some but not all implementations, services can be provided in a cloud computing environment, for example, in the context of a multi-tenant database system. Thus, some of the disclosed techniques can be implemented without having to install software locally, that is, on computing devices of users interacting with services available through the cloud. Some of the disclosed techniques can be implemented via an application installed on computing devices of users.

[0044] Information stored in a database record can include various types of data including character-based data, audio data, image data, animated images, and/or video data. A database record can store one or more files, which can include text, presentations, documents, multimedia files, and the like. Data retrieved from a database can be presented via a computing device. For example, visual data can be displayed in a graphical user interface (GUI) on a display device such as the display of the computing device. In some but not all implementations, the disclosed methods, apparatus, systems, and computer program products may be configured or designed for use in a multi-tenant database environment.

[0045] The term "multi-tenant database system" generally refers to those systems in which various elements of hardware and/or software of a database system may be shared by one or more customers. For example, a given application server may simultaneously process requests for a great number of customers, and a given database table may store rows of data such as feed items for a potentially much greater number of customers.

[0046] An example of a "user profile" or "user's profile" is a database object or set of objects configured to store and maintain data about a given user of a social networking system and/or database system. The data can include general information, such as name, title, phone number, a photo, a biographical summary, and a status, e.g., text describing what the user is currently doing. Where there are multiple tenants, a user is typically associated with a particular tenant. For example, a user could be a salesperson of a company, which is a tenant of the database system that provides a database service.

[0047] The term "record" generally refers to a data entity having fields with values and stored in database system. An example of a record is an instance of a data object created by a user of the database service, for example, in the form of a CRM record about a particular (actual or potential) business relationship or project. The record can have a data structure defined by the database service (a standard object) or defined by a user (custom object). For example, a record can be for a business partner or potential business partner (e.g., a client, vendor, distributor, etc.) of the user, and can include information describing an entire company, subsidiaries, or contacts at the company. As another example, a record can be a project that the user is working on, such as an opportunity (e.g., a possible sale) with an existing partner, or a project that the user is trying to get. In one implementation of a multi-tenant database system, each record for the tenants has a unique identifier stored in a common table. A record has data fields that are defined by the structure of the object (e.g., fields of certain data types and purposes). A record can also have custom fields defined by a user. A field can be another record or include links thereto, thereby providing a parent-child relationship between the records.

[0048] Some non-limiting examples of systems, apparatus, and methods are described below for implementing database systems and enterprise level social networking systems in conjunction with the disclosed techniques. Such implementations can provide more efficient use of a database system. For instance, a user of a database system may not easily know when important information in the database has changed, e.g., about a project or client. Such implementations can provide feed tracked updates about such changes and other events, thereby keeping users informed.

[0049] FIG. **6**A shows a block diagram of an example of an environment **10** in which an on-demand database service exists and can be used in accordance with some implementations. Environment **10** may include user systems **12**, network **14**, database system **16**, processor system **17**, application platform **18**, network interface **20**, tenant data storage **22**, system data storage **24**, program code **26**, and process space **28**. In other implementations, environment **10** may not have all of these components and/or may have other components instead of, or in addition to, those listed above.

[0050] A user system **12** may be implemented as any computing device(s) or other data processing apparatus such as a machine or system used by a user to access a database system **16**. For example, any of user systems **12** can be a handheld and/or portable computing device such as a mobile phone, a smartphone, a laptop computer, or a tablet. Other examples of a user system include computing devices such as a work station and/or a network of computing devices. As illustrated in FIG. **6**A (and in more detail in FIG. **6**B) user systems **12** might interact via a network **14** with an on-demand database service, which is implemented in the example of FIG. **6**A as database system **16**.

[0051] An on-demand database service, implemented using system **16** by way of example, is a service that is made available to users who do not need to necessarily be concerned with building and/or maintaining the database system. Instead, the database system may be available for their use when the users need the database system, i.e., on the demand of the users. Some on-demand database services may store information from one or more tenants into tables of a common database image to form a multi-tenant database system (MTS). A database image may include one or more database objects. A relational database management system (RDBMS) or the equivalent may execute storage and retrieval of information against the database object(s). Application platform **18** may be a framework that allows the applications of system **16** to run, such as the hardware and/or software, e.g., the operating system. In some implementations, application platform **18** enables creation, managing and executing one or more applications developed by the provider of the on-demand database service, users accessing the on-demand database service via user systems **12**, or third party application developers accessing the on-demand database service via user systems **12**.

[0052] The users of user systems **12** may differ in their respective capacities, and the capacity of a particular user system **12** might be entirely determined by permissions (permission levels) for the current user. For example, when a salesperson is using a particular user system **12** to interact with system **16**, the user system has the capacities allotted to that salesperson. However, while an administrator is using that user system to interact with system **16**, that user system

has the capacities allotted to that administrator. In systems with a hierarchical role model, users at one permission level may have access to applications, data, and database information accessible by a lower permission level user, but may not have access to certain applications, database information, and data accessible by a user at a higher permission level. Thus, different users will have different capabilities with regard to accessing and modifying application and database information, depending on a user's security or permission level, also called authorization.

[0053] Network **14** is any network or combination of networks of devices that communicate with one another. For example, network **14** can be any one or any combination of a LAN (local area network), WAN (wide area network), telephone network, wireless network, point-to-point network, star network, token ring network, hub network, or other appropriate configuration. Network **14** can include a TCP/IP (Transfer Control Protocol and Internet Protocol) network, such as the global internetwork of networks often referred to as the Internet. The Internet will be used in many of the examples herein. However, it should be understood that the networks that the present implementations might use are not so limited.

[0054] User systems **12** might communicate with system **16** using TCP/IP and, at a higher network level, use other common Internet protocols to communicate, such as HTTP, FTP, AFS, WAP, etc. In an example where HTTP is used, user system **12** might include an HTTP client commonly referred to as a "browser" for sending and receiving HTTP signals to and from an HTTP server at system **16**. Such an HTTP server might be implemented as the sole network interface **20** between system **16** and network **14**, but other techniques might be used as well or instead. In some implementations, the network interface **20** between system **16** and network **14** includes load sharing functionality, such as round-robin HTTP request distributors to balance loads and distribute incoming HTTP requests evenly over a plurality of servers. At least for users accessing system **16**, each of the plurality of servers has access to the MTS' data; however, other alternative configurations may be used instead.

[0055] In one implementation, system **16**, shown in FIG. **6**A, implements a web-based CRM system. For example, in one implementation, system **16** includes application servers configured to implement and execute CRM software applications as well as provide related data, code, forms, web pages and other information to and from user systems **12** and to store to, and retrieve from, a database system related data, objects, and Webpage content. With a multi-tenant system, data for multiple tenants may be stored in the same physical database object in tenant data storage **22**, however, tenant data typically is arranged in the storage medium(s) of tenant data storage **22** so that data of one tenant is kept logically separate from that of other tenants so that one tenant does not have access to another tenant's data, unless such data is expressly shared. In certain implementations, system **16** implements applications other than, or in addition to, a CRM application. For example, system **16** may provide tenant access to multiple hosted (standard and custom) applications, including a CRM application. User (or third party developer) applications, which may or may not include CRM, may be supported by the application platform **18**, which manages creation, storage of the applications into one or more database objects and executing of the applications in a virtual machine in the process space of the system **16**.

[0056] One arrangement for elements of system **16** is shown in FIGS. **7**A and **7**B, including a network interface **20**, application platform **18**, tenant data storage **22** for tenant data **23**, system data storage **24** for system data **25** accessible to system **16** and possibly multiple tenants, program code **26** for implementing various functions of system **16**, and a process space **28** for executing MTS system processes and tenant-specific processes, such as running applications as part of an application hosting service. Additional processes that may execute on system **16** include database indexing processes.

[0057] Several elements in the system shown in FIG. **6**A include conventional, well-known elements that are explained only briefly here. For example, each user system **12** could include a desktop personal computer, workstation, laptop, PDA, cell phone, or any wireless access protocol (WAP) enabled device or any other computing device capable of interfacing directly or indirectly to the Internet or other network connection. The term "computing device" is also referred to herein simply as a "computer". User system **12** typically runs an HTTP client, e.g., a browsing program, such as Microsoft's Internet Explorer browser, Netscape's Navigator browser, Opera's browser, or a WAP-enabled browser in the case of a cell phone, PDA or other wireless device, or the like, allowing a user (e.g., subscriber of the multi-tenant database system) of user system **12** to access, process and view information, pages and applications available to it from system **16** over network **14**. Each user system **12** also typically includes one or more user input devices, such as a keyboard, a mouse, trackball, touch pad, touch screen, pen or the like, for interacting with a GUI provided by the browser on a display (e.g., a monitor screen, LCD display, OLED display, etc.) of the computing device in conjunction with pages, forms, applications and other information provided by system **16** or other systems or servers. Thus, "display device" as used herein can refer to a display of a computer system such as a monitor or touch-screen display, and can refer to any computing device having display capabilities such as a desktop computer, laptop, tablet, smartphone, a television set-top box, or wearable device such Google Glass® or other human body-mounted display apparatus. For example, the display device can be used to access data and applications hosted by system **16**, and to perform searches on stored data, and otherwise allow a user to interact with various GUI pages that may be presented to a user. As discussed above, implementations are suitable for use with the Internet, although other networks can be used instead of or in addition to the Internet, such as an intranet, an extranet, a virtual private network (VPN), a non-TCP/IP based network, any LAN or WAN or the like.

[0058] According to one implementation, each user system **12** and all of its components are operator configurable using applications, such as a browser, including computer code run using a central processing unit such as an Intel Pentium® processor or the like. Similarly, system **16** (and additional instances of an MTS, where more than one is present) and all of its components might be operator configurable using application(s) including computer code to run using processor system **17**, which may be implemented to include a central processing unit, which may include an Intel Pentium® processor or the like, and/or multiple processor units. Non-transitory computer-readable media can

have instructions stored thereon/in, that can be executed by or used to program a computing device to perform any of the methods of the implementations described herein. Computer program code **26** implementing instructions for operating and configuring system **16** to intercommunicate and to process web pages, applications and other data and media content as described herein is preferably downloadable and stored on a hard disk, but the entire program code, or portions thereof, may also be stored in any other volatile or non-volatile memory medium or device as is well known, such as a ROM or RAM, or provided on any media capable of storing program code, such as any type of rotating media including floppy disks, optical discs, digital versatile disk (DVD), compact disk (CD), microdrive, and magneto-optical disks, and magnetic or optical cards, nanosystems (including molecular memory ICs), or any other type of computer-readable medium or device suitable for storing instructions and/or data. Additionally, the entire program code, or portions thereof, may be transmitted and downloaded from a software source over a transmission medium, e.g., over the Internet, or from another server, as is well known, or transmitted over any other conventional network connection as is well known (e.g., extranet, VPN, LAN, etc.) using any communication medium and protocols (e.g., TCP/IP, HTTP, HTTPS, Ethernet, etc.) as are well known. It will also be appreciated that computer code for the disclosed implementations can be realized in any programming language that can be executed on a client system and/or server or server system such as, for example, C, C++, HTML, any other markup language, Java™, JavaScript, ActiveX, any other scripting language, such as VBScript, and many other programming languages as are well known may be used. (Java™ is a trademark of Sun Microsystems, Inc.).

[0059] According to some implementations, each system **16** is configured to provide web pages, forms, applications, data and media content to user (client) systems **12** to support the access by user systems **12** as tenants of system **16**. As such, system **16** provides security mechanisms to keep each tenant's data separate unless the data is shared. If more than one MTS is used, they may be located in close proximity to one another (e.g., in a server farm located in a single building or campus), or they may be distributed at locations remote from one another (e.g., one or more servers located in city A and one or more servers located in city B). As used herein, each MTS could include one or more logically and/or physically connected servers distributed locally or across one or more geographic locations. Additionally, the term "server" is meant to refer to one type of computing device such as a system including processing hardware and process space(s), an associated storage medium such as a memory device or database, and, in some instances, a database application (e.g., OODBMS or RDBMS) as is well known in the art. It should also be understood that "server system" and "server" are often used interchangeably herein. Similarly, the database objects described herein can be implemented as single databases, a distributed database, a collection of distributed databases, a database with redundant online or offline backups or other redundancies, etc., and might include a distributed database or storage network and associated processing intelligence.

[0060] FIG. **6**B shows a block diagram of an example of some implementations of elements of FIG. **6**A and various possible interconnections between these elements. That is, FIG. **6**B also illustrates environment **10**. However, in FIG. **6**B elements of system **16** and various interconnections in some implementations are further illustrated. FIG. **6**B shows that user system **12** may include processor system **12**A, memory system **12**B, input system **12**C, and output system **12**D. FIG. **6**B shows network **14** and system **16**. FIG. **6**B also shows that system **16** may include tenant data storage **22**, tenant data **23**, system data storage **24**, system data **25**, User Interface (UI) **30**, Application Program Interface (API) **32**, PL/SOQL **34**, save routines **36**, application setup mechanism **38**, application servers $\mathbf{50}_1$-$\mathbf{50}_N$, system process space **52**, tenant process spaces **54**, tenant management process space **60**, tenant storage space **62**, user storage **64**, and application metadata **66**. In other implementations, environment **10** may not have the same elements as those listed above and/or may have other elements instead of, or in addition to, those listed above.

[0061] User system **12**, network **14**, system **16**, tenant data storage **22**, and system data storage **24** were discussed above in FIG. **6**A. Regarding user system **12**, processor system **12**A may be any combination of one or more processors. Memory system **12**B may be any combination of one or more memory devices, short term, and/or long term memory. Input system **12**C may be any combination of input devices, such as one or more keyboards, mice, trackballs, scanners, cameras, and/or interfaces to networks. Output system **12**D may be any combination of output devices, such as one or more monitors, printers, and/or interfaces to networks. As shown by FIG. **6**B, system **16** may include a network interface **20** (of FIG. **6**A) implemented as a set of application servers **50**, an application platform **18**, tenant data storage **22**, and system data storage **24**. Also shown is system process space **52**, including individual tenant process spaces **54** and a tenant management process space **60**. Each application server **50** may be configured to communicate with tenant data storage **22** and the tenant data **23** therein, and system data storage **24** and the system data **25** therein to serve requests of user systems **12**. The tenant data **23** might be divided into individual tenant storage spaces **62**, which can be either a physical arrangement and/or a logical arrangement of data. Within each tenant storage space **62**, user storage **64** and application metadata **66** might be similarly allocated for each user. For example, a copy of a user's most recently used (MRU) items might be stored to user storage **64**. Similarly, a copy of MRU items for an entire organization that is a tenant might be stored to tenant storage space **62**. A UI **30** provides a user interface and an API **32** provides an application programmer interface to system **16** resident processes to users and/or developers at user systems **12**. The tenant data and the system data may be stored in various databases, such as one or more Oracle® databases.

[0062] Application platform **18** includes an application setup mechanism **38** that supports application developers' creation and management of applications, which may be saved as metadata into tenant data storage **22** by save routines **36** for execution by subscribers as one or more tenant process spaces **54** managed by tenant management process **60** for example. Invocations to such applications may be coded using PL/SQL **34** that provides a programming language style interface extension to API **32**. A detailed description of some PL/SOQL language implementations is discussed in commonly assigned U.S. Pat. No. 7,730,478, titled METHOD AND SYSTEM FOR ALLOWING ACCESS TO DEVELOPED APPLICATIONS VIA A MULTI-TENANT ON-DEMAND DATABASE SERVICE,

by Craig Weissman, issued on Jun. 1, 2010, and hereby incorporated by reference in its entirety and for all purposes. Invocations to applications may be detected by one or more system processes, which manage retrieving application metadata **66** for the subscriber making the invocation and executing the metadata as an application in a virtual machine.

[0063] Each application server **50** may be communicably coupled to database systems, e.g., having access to system data **25** and tenant data **23**, via a different network connection. For example, one application server $\mathbf{50}_1$ might be coupled via the network **14** (e.g., the Internet), another application server $\mathbf{50}_{N-1}$ might be coupled via a direct network link, and another application server $\mathbf{50}_N$ might be coupled by yet a different network connection. Transfer Control Protocol and Internet Protocol (TCP/IP) are typical protocols for communicating between application servers **50** and the database system. However, it will be apparent to one skilled in the art that other transport protocols may be used to optimize the system depending on the network interconnect used.

[0064] In certain implementations, each application server **50** is configured to handle requests for any user associated with any organization that is a tenant. Because it is desirable to be able to add and remove application servers from the server pool at any time for any reason, there is preferably no server affinity for a user and/or organization to a specific application server **50**. In one implementation, therefore, an interface system implementing a load balancing function (e.g., an F5 Big-IP load balancer) is communicably coupled between the application servers **50** and the user systems **12** to distribute requests to the application servers **50**. In one implementation, the load balancer uses a least connections algorithm to route user requests to the application servers **50**. Other examples of load balancing algorithms, such as round robin and observed response time, also can be used. For example, in certain implementations, three consecutive requests from the same user could hit three different application servers **50**, and three requests from different users could hit the same application server **50**. In this manner, by way of example, system **16** is multi-tenant, wherein system **16** handles storage of, and access to, different objects, data and applications across disparate users and organizations.

[0065] As an example of storage, one tenant might be a company that employs a sales force where each salesperson uses system **16** to manage their sales process. Thus, a user might maintain contact data, leads data, customer follow-up data, performance data, goals and progress data, etc., all applicable to that user's personal sales process (e.g., in tenant data storage **22**). In an example of a MTS arrangement, since all of the data and the applications to access, view, modify, report, transmit, calculate, etc., can be maintained and accessed by a user system having nothing more than network access, the user can manage his or her sales efforts and cycles from any of many different user systems. For example, if a salesperson is visiting a customer and the customer has Internet access in their lobby, the salesperson can obtain critical updates as to that customer while waiting for the customer to arrive in the lobby.

[0066] While each user's data might be separate from other users' data regardless of the employers of each user, some data might be organization-wide data shared or accessible by a plurality of users or all of the users for a given organization that is a tenant. Thus, there might be some data structures managed by system **16** that are allocated at the tenant level while other data structures might be managed at the user level. Because an MTS might support multiple tenants including possible competitors, the MTS should have security protocols that keep data, applications, and application use separate. Also, because many tenants may opt for access to an MTS rather than maintain their own system, redundancy, up-time, and backup are additional functions that may be implemented in the MTS. In addition to user-specific data and tenant-specific data, system **16** might also maintain system level data usable by multiple tenants or other data. Such system level data might include industry reports, news, postings, and the like that are sharable among tenants.

[0067] In certain implementations, user systems **12** (which may be client systems) communicate with application servers **50** to request and update system-level and tenant-level data from system **16** that may involve sending one or more queries to tenant data storage **22** and/or system data storage **24**. System **16** (e.g., an application server **50** in system **16**) automatically generates one or more SQL statements (e.g., one or more SQL queries) that are designed to access the desired information. System data storage **24** may generate query plans to access the requested data from the database.

[0068] Each database can generally be viewed as a collection of objects, such as a set of logical tables, containing data fitted into predefined categories. A "table" is one representation of a data object, and may be used herein to simplify the conceptual description of objects and custom objects according to some implementations. It should be understood that "table" and "object" may be used interchangeably herein. Each table generally contains one or more data categories logically arranged as columns or fields in a viewable schema. Each row or record of a table contains an instance of data for each category defined by the fields. For example, a CRM database may include a table that describes a customer with fields for basic contact information such as name, address, phone number, fax number, etc. Another table might describe a purchase order, including fields for information such as customer, product, sale price, date, etc. In some multi-tenant database systems, standard entity tables might be provided for use by all tenants. For CRM database applications, such standard entities might include tables for case, account, contact, lead, and opportunity data objects, each containing pre-defined fields. It should be understood that the word "entity" may also be used interchangeably herein with "object" and "table".

[0069] In some multi-tenant database systems, tenants may be allowed to create and store custom objects, or they may be allowed to customize standard entities or objects, for example by creating custom fields for standard objects, including custom index fields. Commonly assigned U.S. Pat. No. 7,779,039, titled CUSTOM ENTITIES AND FIELDS IN A MULTI-TENANT DATABASE SYSTEM, by Weissman et al., issued on Aug. 17, 2010, and hereby incorporated by reference in its entirety and for all purposes, teaches systems and methods for creating custom objects as well as customizing standard objects in a multi-tenant database system. In certain implementations, for example, all custom entity data rows are stored in a single multi-tenant physical table, which may contain multiple logical tables per organization. It is transparent to customers that their multiple

"tables" are in fact stored in one large table or that their data may be stored in the same table as the data of other customers.

[0070] FIG. **7**A shows a system diagram of an example of architectural components of an on-demand database service environment **900**, in accordance with some implementations. A client machine located in the cloud **904**, generally referring to one or more networks in combination, as described herein, may communicate with the on-demand database service environment via one or more edge routers **908** and **912**. A client machine can be any of the examples of user systems **12** described above. The edge routers may communicate with one or more core switches **920** and **924** via firewall **916**. The core switches may communicate with a load balancer **928**, which may distribute server load over different pods, such as the pods **940** and **944**. The pods **940** and **944**, which may each include one or more servers and/or other computing resources, may perform data processing and other operations used to provide on-demand services. Communication with the pods may be conducted via pod switches **932** and **936**. Components of the on-demand database service environment may communicate with a database storage **956** via a database firewall **948** and a database switch **952**.

[0071] As shown in FIGS. **7**A and **7**B, accessing an on-demand database service environment may involve communications transmitted among a variety of different hardware and/or software components. Further, the on-demand database service environment **900** is a simplified representation of an actual on-demand database service environment. For example, while only one or two devices of each type are shown in FIGS. **7**A and **7**B, some implementations of an on-demand database service environment may include anywhere from one to many devices of each type. Also, the on-demand database service environment need not include each device shown in FIGS. **7**A and **7**B, or may include additional devices not shown in FIGS. **7**A and **7**B.

[0072] Moreover, one or more of the devices in the on-demand database service environment **900** may be implemented on the same physical device or on different hardware. Some devices may be implemented using hardware or a combination of hardware and software.

[0073] Thus, terms such as "data processing apparatus," "machine," "server" and "device" as used herein are not limited to a single hardware device, but rather include any hardware and software configured to provide the described functionality.

[0074] The cloud **904** is intended to refer to a data network or combination of data networks, often including the Internet. Client machines located in the cloud **904** may communicate with the on-demand database service environment to access services provided by the on-demand database service environment. For example, client machines may access the on-demand database service environment to retrieve, store, edit, and/or process information.

[0075] In some implementations, the edge routers **908** and **912** route packets between the cloud **904** and other components of the on-demand database service environment **900**. The edge routers **908** and **912** may employ the Border Gateway Protocol (BGP). The BGP is the core routing protocol of the Internet. The edge routers **908** and **912** may maintain a table of IP networks or 'prefixes', which designate network reachability among autonomous systems on the Internet.

[0076] In one or more implementations, the firewall **916** may protect the inner components of the on-demand database service environment **900** from Internet traffic. The firewall **916** may block, permit, or deny access to the inner components of the on-demand database service environment **900** based upon a set of rules and other criteria. The firewall **916** may act as one or more of a packet filter, an application gateway, a stateful filter, a proxy server, or any other type of firewall.

[0077] In some implementations, the core switches **920** and **924** are high-capacity switches that transfer packets within the on-demand database service environment **900**. The core switches **920** and **924** may be configured as network bridges that quickly route data between different components within the on-demand database service environment. In some implementations, the use of two or more core switches **920** and **924** may provide redundancy and/or reduced latency.

[0078] In some implementations, the pods **940** and **944** may perform the core data processing and service functions provided by the on-demand database service environment. Each pod may include various types of hardware and/or software computing resources. An example of the pod architecture is discussed in greater detail with reference to FIG. **7**B.

[0079] In some implementations, communication between the pods **940** and **944** may be conducted via the pod switches **932** and **936**. The pod switches **932** and **936** may facilitate communication between the pods **940** and **944** and client machines located in the cloud **904**, for example via core switches **920** and **924**. Also, the pod switches **932** and **936** may facilitate communication between the pods **940** and **944** and the database storage **956**.

[0080] In some implementations, the load balancer **928** may distribute workload between the pods **940** and **944**. Balancing the on-demand service requests between the pods may assist in improving the use of resources, increasing throughput, reducing response times, and/or reducing overhead. The load balancer **928** may include multilayer switches to analyze and forward traffic.

[0081] In some implementations, access to the database storage **956** may be guarded by a database firewall **948**. The database firewall **948** may act as a computer application firewall operating at the database application layer of a protocol stack. The database firewall **948** may protect the database storage **956** from application attacks such as structure query language (SQL) injection, database rootkits, and unauthorized information disclosure.

[0082] In some implementations, the database firewall **948** may include a host using one or more forms of reverse proxy services to proxy traffic before passing it to a gateway router. The database firewall **948** may inspect the contents of database traffic and block certain content or database requests. The database firewall **948** may work on the SQL application level atop the TCP/IP stack, managing applications' connection to the database or SQL management interfaces as well as intercepting and enforcing packets traveling to or from a database network or application interface.

[0083] In some implementations, communication with the database storage **956** may be conducted via the database switch **952**. The multi-tenant database storage **956** may include more than one hardware and/or software components for handling database queries. Accordingly, the data-

base switch **952** may direct database queries transmitted by other components of the on-demand database service environment (e.g., the pods **940** and **944**) to the correct components within the database storage **956**.

[0084] In some implementations, the database storage **956** is an on-demand database system shared by many different organizations. The on-demand database service may employ a multi-tenant approach, a virtualized approach, or any other type of database approach. On-demand database services are discussed in greater detail with reference to FIGS. **7**A and **7**B.

[0085] FIG. **7**B shows a system diagram further illustrating an example of architectural components of an on-demand database service environment, in accordance with some implementations. The pod **944** may be used to render services to a user of the on-demand database service environment **900**. In some implementations, each pod may include a variety of servers and/or other systems. The pod **944** includes one or more content batch servers **964**, content search servers **968**, query servers **982**, file servers **986**, access control system (ACS) servers **980**, batch servers **984**, and app servers **988**. Also, the pod **944** includes database instances **990**, quick file systems (QFS) **992**, and indexers **994**. In one or more implementations, some or all communication between the servers in the pod **944** may be transmitted via the switch **936**.

[0086] The content batch servers **964** may handle requests internal to the pod. These requests may be long-running and/or not tied to a particular customer. For example, the content batch servers **964** may handle requests related to log mining, cleanup work, and maintenance tasks.

[0087] The content search servers **968** may provide query and indexer functions. For example, the functions provided by the content search servers **968** may allow users to search through content stored in the on-demand database service environment.

[0088] The file servers **986** may manage requests for information stored in the file storage **998**. The file storage **998** may store information such as documents, images, and basic large objects (BLOBs). By managing requests for information using the file servers **986**, the image footprint on the database may be reduced.

[0089] The query servers **982** may be used to retrieve information from one or more file systems. For example, the query system **982** may receive requests for information from the app servers **988** and then transmit information queries to the NFS **996** located outside the pod.

[0090] The pod **944** may share a database instance **990** configured as a multi-tenant environment in which different organizations share access to the same database. Additionally, services rendered by the pod **944** may call upon various hardware and/or software resources. In some implementations, the ACS servers **980** may control access to data, hardware resources, or software resources.

[0091] In some implementations, the batch servers **984** may process batch jobs, which are used to run tasks at specified times. Thus, the batch servers **984** may transmit instructions to other servers, such as the app servers **988**, to trigger the batch jobs.

[0092] In some implementations, the QFS **992** may be an open source file system available from Sun Microsystems® of Santa Clara, California. The QFS may serve as a rapid-access file system for storing and accessing information available within the pod **944**. The QFS **992** may support some volume management capabilities, allowing many disks to be grouped together into a file system. File system metadata can be kept on a separate set of disks, which may be useful for streaming applications where long disk seeks cannot be tolerated. Thus, the QFS system may communicate with one or more content search servers **968** and/or indexers **994** to identify, retrieve, move, and/or update data stored in the network file systems **996** and/or other storage systems.

[0093] In some implementations, one or more query servers **982** may communicate with the NFS **996** to retrieve and/or update information stored outside of the pod **944**. The NFS **996** may allow servers located in the pod **944** to access information to access files over a network in a manner similar to how local storage is accessed.

[0094] In some implementations, queries from the query servers **922** may be transmitted to the NFS **996** via the load balancer **928**, which may distribute resource requests over various resources available in the on-demand database service environment. The NFS **996** may also communicate with the QFS **992** to update the information stored on the NFS **996** and/or to provide information to the QFS **992** for use by servers located within the pod **944**.

[0095] In some implementations, the pod may include one or more database instances **990**.

[0096] The database instance **990** may transmit information to the QFS **992**. When information is transmitted to the QFS, it may be available for use by servers within the pod **944** without using an additional database call.

[0097] In some implementations, database information may be transmitted to the indexer **994**. Indexer **994** may provide an index of information available in the database **990** and/or QFS **992**. The index information may be provided to file servers **986** and/or the QFS **992**.

[0098] In some implementations, one or more application servers or other servers described above with reference to FIGS. **7**A and **7**B include a hardware and/or software framework configurable to execute procedures using programs, routines, scripts, etc. Thus, in some implementations, one or more of application servers $\mathbf{50}_1$-$\mathbf{50}_N$ of FIG. **7**B can be configured to initiate performance of one or more of the operations described above by instructing another computing device to perform an operation. In some implementations, one or more application servers $\mathbf{50}_1$-$\mathbf{50}_N$ carry out, either partially or entirely, one or more of the disclosed operations. In some implementations, app servers **988** of FIG. **7**B support the construction of applications provided by the on-demand database service environment **900** via the pod **944**. Thus, an app server **988** may include a hardware and/or software framework configurable to execute procedures to partially or entirely carry out or instruct another computing device to carry out one or more operations disclosed herein. In alternative implementations, two or more app servers **988** may cooperate to perform or cause performance of such operations. Any of the databases and other storage facilities described above with reference to FIGS. **6**A, **6**B, **7**A and **7**B can be configured to store lists, articles, documents, records, files, and other objects for implementing the operations described above. For instance, lists of available communication channels associated with share actions for sharing a type of data item can be maintained in tenant data storage **22** and/or system data storage **24** of FIGS. **7**A and **7**B. By the same token, lists of default or designated channels for particular share actions can be

maintained in storage **22** and/or storage **24**. In some other implementations, rather than storing one or more lists, articles, documents, records, and/or files, the databases and other storage facilities described above can store pointers to the lists, articles, documents, records, and/or files, which may instead be stored in other repositories external to the systems and environments described above with reference to FIGS. **6**A, **6**B, **7**A and **7**B.

[0099] While some of the disclosed implementations may be described with reference to a system having an application server providing a front end for an on-demand database service capable of supporting multiple tenants, the disclosed implementations are not limited to multi-tenant databases nor deployment on application servers. Some implementations may be practiced using various database architectures such as ORACLE®, DB2® by IBM and the like without departing from the scope of the implementations claimed.

[0100] It should be understood that some of the disclosed implementations can be embodied in the form of control logic using hardware and/or computer software in a modular or integrated manner. Other ways and/or methods are possible using hardware and a combination of hardware and software.

[0101] Any of the disclosed implementations may be embodied in various types of hardware, software, firmware, and combinations thereof. For example, some techniques disclosed herein may be implemented, at least in part, by computer-readable media that include program instructions, state information, etc., for performing various services and operations described herein. Examples of program instructions include both machine code, such as produced by a compiler, and files containing higher-level code that may be executed by a computing device such as a server or other data processing apparatus using an interpreter. Examples of computer-readable media include, but are not limited to: magnetic media such as hard disks, floppy disks, and magnetic tape; optical media such as flash memory, compact disk (CD) or digital versatile disk (DVD); magneto-optical media; and hardware devices specially configured to store program instructions, such as read-only memory (ROM) devices and random access memory (RAM) devices. A computer-readable medium may be any combination of such storage devices.

[0102] Any of the operations and techniques described in this application may be implemented as software code to be executed by a processor using any suitable computer language such as, for example, Java, C++ or Perl using, for example, object-oriented techniques. The software code may be stored as a series of instructions or commands on a computer-readable medium. Computer-readable media encoded with the software/program code may be packaged with a compatible device or provided separately from other devices (e.g., via Internet download). Any such computer-readable medium may reside on or within a single computing device or an entire computer system, and may be among other computer-readable media within a system or network. A computer system or computing device may include a monitor, printer, or other suitable display for providing any of the results mentioned herein to a user.

[0103] While various implementations have been described herein, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of the present application should not be limited by any of the implementations described herein, but should be defined only in accordance with the following and later-submitted claims and their equivalents.

1. A method, comprising:
tracking, by a backup poller in a private subnet, a database backup operation;
detecting, by the backup poller, a failure of the database backup operation at a specific point in the database backup operation;
instructing, by the backup poller, a backup decentralizer in a public subnet, to continue the database backup operation from the specific point;
monitoring, by the backup decentralizer, a health of a plurality of service providers;
selecting, by the backup decentralizer, a service provider of the plurality of service providers based on the health of the service provider; and
continuing, by the backup decentralizer, the backup operation from the specific point using the selected service provider.

2. The method of claim **1**, the plurality of service providers each being in a corresponding public subnet.

3. The method of claim **1**, wherein continuing the backup operation comprises:
storing a portion of a file in a storage target of the service provider.

4. The method of claim **1**, wherein continuing the backup operation comprises:
storing one of a plurality of files in a storage target of the service provider.

5. The method of claim **1**, further comprising:
storing, by the backup poller, storage metadata for the backup operation; and
transmitting, by the backup poller to the backup decentralizer, the storage metadata, wherein the backup decentralizer uses the storage metadata to continue the backup operation.

6. The method of claim **5**, the storage metadata comprising:
a backup transaction identifier (ID).

7. A system comprising:
a server system including a processor and a memory, the server system-configurable to cause;
tracking, by a backup poller in a private subnet, a database backup operation;
detecting, by the backup poller, a failure of the database backup operation at a specific point in the database backup operation;
instructing, by the backup poller, a backup decentralizer in a public subnet, to continue the database backup operation from the specific point;
monitoring, by the backup decentralizer, a health of a plurality of service providers;
selecting, by the backup decentralizer, a service provider of the plurality of service providers based on the health of the service provider; and
continuing, by the backup decentralizer, the backup operation from the specific point using the selected service provider.

8. A non-transitory computer readable medium storing thereon computer-readable program code capable of being executed by one or more processors, the program code comprising computer-readable instructions configurable to cause:

tracking, by a backup poller in a private subnet, a database backup operation;

detecting, by the backup poller, a failure of the database backup operation at a specific point in the database backup operation;

instructing, by the backup poller, a backup decentralizer in a public subnet, to continue the database backup operation from the specific point;

monitoring, by the backup decentralizer, a health of a plurality of service providers;

selecting, by the backup decentralizer, a service provider of the plurality of service providers based on the health of the service provider; and

continuing, by the backup decentralizer, the backup operation from the specific point using the selected service provider.

9. The method of claim **1**, further comprising:

accessing, by the backup poller, backup metadata associated with the database backup operation; and

storing, by the backup poller, the backup metadata in a database.

10. The method of claim **9**, the backup metadata including one or more of: backup identifier (ID), a backup file name, a backup file ID, backup file type, backup file size, backup assigned initial storage provider ID, backup cloud provider, backup cloud provider ID, backup cloud provider storage class, backup cloud provider storage class ID, backup file ID transaction status, an indicator of backup file size, backup file ID split file ID, backup file ID split file ID transaction status, backup retry limit per cloud provider, or a backup retry timeout.

11. The method of claim **1**, the backup decentralizer obtaining backup metadata or portion thereof from the backup poller via an application programming interface of the backup decentralizer.

12. The method of claim **1**, wherein monitoring, by the backup decentralizer, a health of a plurality of service providers comprises:

performing, by the backup decentralizer, a health check by polling one or more of: hardware, network file system, domain name system, or storage service.

13. The method of claim **1**, wherein monitoring, by the backup decentralizer, a health of a plurality of service providers comprises:

performing, by a public application programming interface of the backup decentralizer, a health check on system services that will be engaged in a database backup transaction.

14. The system of claim **7**, the database system further configurable to cause:

storing, by the backup poller, storage metadata for the backup operation; and

transmitting, by the backup poller to the backup decentralizer, the storage metadata, wherein the backup decentralizer uses the storage metadata to continue the backup operation.

15. The system of claim **7**, wherein continuing the backup operation comprises:

storing one of a plurality of files in a storage target of the service provider.

16. The system of claim **7**, wherein continuing the backup operation comprises:

storing a portion of a file in a storage target of the service provider.

17. The system of claim **7**, the plurality of service providers each being in a corresponding public subnet.

18. The non-transitory computer readable medium of claim **8**, the plurality of service providers each being in a corresponding public subnet.

19. The non-transitory computer readable medium of claim **8**, the program code further comprising computer-readable instructions configurable to cause:

storing, by the backup poller, storage metadata for the backup operation; and

transmitting, by the backup poller to the backup decentralizer, the storage metadata, wherein the backup decentralizer uses the storage metadata to continue the backup operation.

20. The non-transitory computer readable medium of claim **8**, wherein continuing the backup operation comprises:

storing a portion of a file in a storage target of the service provider.

* * * * *